

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



SUBJECT: SOFTWARE ARCHITECTURE
PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor:	Vu Quang Dung
Student Name:	Pham Xuan Bach
Student ID:	23010118
Email:	23010118@st.phenikaa-uni.edu.vn
Course class:	CSE703110-1-2-25(N01)

Ha Noi, March 2026

I.	UI/UX DESIGN & FRONTEND ARCHITECTURE.....	4
I.1	Monolith-to-Microservices decomposition:	4
I.2	Layered Architecture	4
I.3	Movie Service.....	5
I.4	Booking Service	5
	I.4.1 Atomic Seat Locking:	5
	I.4.2 Grouped Bookings:	5
	I.4.3 Other alterations:	6
I.5	API Gateway.....	6
	I.5.1 Dual-Factor Authentication:	6
	I.5.2 Role-based Access Control.....	6
	I.5.3 Streaming Proxy for Posters	6
I.6	Payment Service	6
	I.6.1 Integrity and Validation	7
	I.6.2 Succesful Payment:.....	7
I.7	User Service & Authentication.....	7
	I.7.1 Registration.....	7
	I.7.2 Session Management	7
	I.7.3 API Gateway Intergration & Token Verification.....	7
	I.7.4 Admin Control & Seeding	8
I.8	Event-Driven Communication (RabbitMQ).....	8
I.9	Schema Migration.....	8
II.	Frontend Development.....	8
II.1	Custom SPA Router & Route Protection	8
II.2	API client.....	9
II.3	Customer-Facing Pages	9
	II.3.1 Movies List & Detail	10

II.3.2	Core Booking Logic	11
II.3.3	Payment & Post-Booking Workflow.....	12
II.4	Admin Dashboard & Management Pages	13
II.5	Reusable Components	15
II.5.1	Global Navigation.....	16
III.	Dockerization	17
III.1	Docker images	17
III.2	Docker Compose Orchestration	17
III.3	Hot-Reload.....	17
III.4	Database Utilities.....	18
IV.	Testing.....	18
IV.1	Base Class	18
IV.2	Test Orchestrator	18
IV.3	Test Suites (6 modules, 32 test cases)	19

I. UI/UX DESIGN & FRONTEND ARCHITECTURE

I.1 Monolith-to-Microservices decomposition:

The system originally used Monolith architecture. Since Lab 4, it was decomposed into 6 independently deployable services, each running its own Flask process on a dedicated port with its own SQLite database:

Service	Path	Port	Database
Movie	<i>src/movie/app.py</i>	5001	movies.db
Booking	<i>src/booking/app.py</i>	5002	booking.db
Payment	<i>src/payment/app.py</i>	5003	payments.db
User	<i>src/user/app.py</i>	5004	users.db
API	<i>src/gateway/app.py</i>	5005	-
Notification	<i>src/notification/app.py</i>	-	-

Each service enforces the **Database-per-Service** pattern, which means no service reads another's database directly. Cross-service communication is done via synchronous HTTP calls, and later by asynchronous RabbitMQ events.

Note: API was originally located at port 5000, but was changed later to prevent network conflict.

I.2 Layered Architecture

Movie and Booking services follow a strict 3-layer architecture:

- Presentation layer: Utilizes Flask to define RESTful endpoints, parse incoming JSON requests, and manage HTTP status codes.
- Business Logic layer: Responsible for validation and orchestration, cross-service API calls, and publishing events to the message broker.

- Persistence layer: Manages direct data access using the *sqlite3* library. This layer executes raw SQL queries and handles dynamic schema migrations to ensure database evolution without service interruption.

Note: Data is passed between these layers using a Model Layer, which includes *to_dict()* methods for standardized JSON serialization across the system.

I.3 Movie Service

Implemented full CRUD for Movies, Showtimes and Rooms:

- Movies: ***POST/GET/PUT/DELETE /api/movies***, search by title via *?query=* parameter, *image_url* and description fields with DB migration support.
- Showtimes: CRUD with duration validation – the service rejects showtimes where the time slot is shorter than the movie’s runtime (***validate_showtime_duration***)
- Rooms: Seeded default room, queried by Booking service for seat layout.
- Poster upload: Implemented file upload endpoint for movie poster (***POST /api/movies/upload***) with extension whitelist, served via Flask *static_folder*.

Note: To simplify, deleting a movie triggers refund + cancel for all associated showtimes’ bookings via internal API to Booking service, then sends ***REFUND_NOTIFICATION*** events to RabbitMQ.

I.4 Booking Service

To handle concurrent users and complex pricing logic, several key architectural patterns and algorithms were implemented:

I.4.1 Atomic Seat Locking:

To comply with the ASR 2 (preventing double-bookings), the system avoids relying on application-level checks which are prone to race conditions. Instead, it utilizes database-level:

```
UPDATE seats SET status = "booked"
WHERE showtime_id = ? AND seat_number = ? AND status = "available"
```

If *rowcount == 0*, the seat was already taken – the entire transaction rolls back. This prevents race conditions without external locks.

I.4.2 Grouped Bookings:

This feature is added in later stages of development. Multiple seats are stored as a comma-separated string under one *booking_id* (e.g., “E5, E6, E7”) rather than creating separate rows per seat. This simplifies the payment flow (one payment per booking for each seat before).

I.4.3 Other alterations:

- Several seat types were implemented: *STANDARD*, *VIP*, and *COUPLE*. Pricing is calculated dynamically at runtime. The Booking Service fetches the base showtime price via an internal REST call to the Movie Service, then iterates through the selected seats, querying the local *seats* table to aggregate any applicable surcharges before finalizing the *total_booking_amount*.
- To optimize database performance and save storage, seats are not pre-generated when showtime is created. Instead, the service employs a lazy Initialization pattern: When a user first requests to view the seat map for showtime, the Booking Service fetches the room configuration (rows and columns) from the Movie Service and generates the physical seat layout in the local database just-in-time.

I.5 API Gateway

Single entry point (*gateway/app.py*) implementing:

I.5.1 Dual-Factor Authentication:

Implemented a two-tier verification process:

- 1, System-level (API key): Every request must include a static Key in the header to prevent unauthorized bot traffic or direct service exposure.
2. User-Level (JWT Proxy): For protected routes, the Gateway extracts the Bearer token and performs a synchronous verification check with the User Service. Upon success, it enriches the request with *X-User-Email* and *X-User-Role* headers before forwarding the payload to the target microservice.

I.5.2 Role-based Access Control

- Public Routes: Catalog viewing (Movies, Showtimes) is accessible via standard API key verification.
- Customer Routes: Booking and Payment operations require valid User JWT.
- Administrative Routes: Management operations (CRUD on Movies/Showtimes, User Management) are strictly restricted to accounts with the *admin* role, verified via the *validate_and_forward* middleware.

I.5.3 Streaming Proxy for Posters

To handle high-bandwidth assets like movie posters, the Gateway implements a Streaming Proxy. Rather than loading entire image files into the Gateway's memory-which could lead to performance bottlenecks, the system utilizes *iter_content* to stream binary data in small chunks (1KB) directly from the Movie Service to the client. This approach ensures low memory overhead and high concurrency support.

I.6 Payment Service

For the purposes of this project, the actual financial authorization with external providers is simulated. This service, while it does not perform real-money transactions, fully implements the internal state machine required.

I.6.1 Integrity and Validation

To prevent fraudulent or mismatched payments, the service implements a strict verification flow. When a payment request is received, the service does not rely on the client-provided amount. Instead, it performs a backend-to-backend call to the Booking Service to fetch the authoritative *amount* and *customer_email* associated with the *booking_id*. Only once these details are verified does the service proceed to finalize the transaction.

I.6.2 Successful Payment:

First, the system issues a synchronous ***PUT /api/bookings/<id>/status*** request to the Booking Service to transition the booking status from ***PENDING_PAYMENT*** to ***confirmed***.

Then, it published an ***INVOICE_PRINT*** event to RabbitMQ. This allows the Notification service to handle receipt generation and delivery asynchronously.

I.7 User Service & Authentication

I.7.1 Registration

Instead of storing plaintext passwords, the service utilized *werkzeug.security* library. Specifically:

- Registration: Passwords are processed through ***generate_password_hash*** during the account creation phase.
- Verification: During login, ***check_password_hash*** is used to compare incoming credentials against the stored hashes, preventing credential leaks.

I.7.2 Session Management

Upon successful login attempt:

- The service generates a unique UUID v4 token.
- This token persisted in a specialized tokens table within the database, mapping the session to the user's email and role.

This approach allows for granular control over active sessions and simplifies the logout/invalidation process compared to stateless alternatives.

I.7.3 API Gateway Intergration & Token Verification

A dedicated verification endpoint, ***GET /api/auth/verify***, is implemented to support the API Gateway. When a request reaches the Gateway with an authorization header, the Gateway performs an internal call to this endpoint to validate the token's existence and

retrieve the associated user context (Email and Role). This ensures that downstream services can trust the identity headers injected by Gateway.

I.7.4 Admin Control & Seeding

To demonstrate, the system is seeded with a default administrator account. Furthermore, privileged routes are provided, allowing users/movies/bookings management.

I.8 Event-Driven Communication (RabbitMQ)

The system utilizes 4 distinct event types distributed across 3 dedicated queues:

Queue	Event	Producer Service	Trigger
<i>ticket_events</i>	<i>OrderPlacedEvent</i>	Booking	Seat booked
<i>invoice_events</i>	<i>INVOICE_PRINT</i>	Payment	Payment processed
<i>notification_events</i>	<i>SHOWTIME_CHANGED</i>	Movie	Showtime time updated
<i>notification_events</i>	<i>REFUND_NOTIFICATION</i>	Movie	Showtime/Movie deleted/price changed

The Notification service is a pure consumer – it subscribes to all 3 queues and logs simulated email/printer output per event type.

To handle network instability or delayed startup of the RabbitMQ broker, the connection uses a retry loop with 5s backoff, and a URL-first pattern (*RABBITMQ_URL* env var, fallback to constructed *amqp://* URL) across all producers.

I.9 Schema Migration

All repositories use *PRAGMA table_info* to check for missing columns and apply *ALTER TABLE* at startup, allowing the app to run on older databases without manual migration scripts.

II. Frontend Development

The frontend is a **Vanilla JavaScript SPA** (no React/Vue/Angular) built with Vite, using Bootstrap 5 for styling. It consists of 15 pages, 2 reusable components, and a centralized API client.

II.1 Custom SPA Router & Route Protection

Implemented a client-side router from scratch using the HTML5 History API. Key features include:

- A *parseUrl()* function decomposes the current path into *resource*, *id*, and *verb* segments, allowing for structured navigation logic.
- Support dynamic *:id* parameters (e.g., */movies/:id*)

- A centralized click listener on *document.body* intercepts all internal *<a>* tag clicks. It prevents the default browser reload behavior and triggers a *pushState* navigation instead, maintaining the SPA state.
- Strips trailing slashes from URLs to ensure consistent route matching and avoid duplicate content issues.

Each page module follows a 2-phase lifecycle

- 1, *render()* returns an HTML string (can be **async** for data fetching)
- 2, *afterRender()* attaches event listeners and runs dynamic logic after DOM insertion.

Admin routes (*/admin/**) are guarded client-side: the router checks *localStorage* for a user with *role=== 'admin'* before rendering. Non-admin users are redirected to the normal login page. The *Header* component is hidden on admin routes, replaced by *AdminSidebar*.

II.2 API client

The core of the client is a *request()* function that wraps the native fetch API. For every outgoing request, it automatically retrieves credentials from the browser's *localStorage* and injects the following mandatory security headers:

- peko-key: A static API key required by the Gateway for basic traffic validation.
- Authorization: A Bearer token representing the user's active session.
- X-User-Email: Authenticated user's email, ensuring consistent identity tracking across microservices.

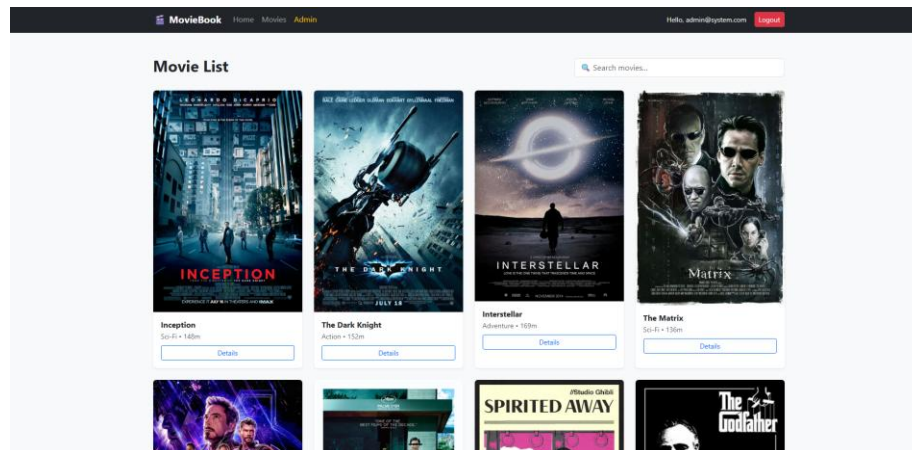
request() also serves as global middleware, providing unified behavior for common API scenarios:

- Session Management: If a service returns a *401 Unauthorized* status (indicating an expired or invalid token), the client automatically clears the local session and redirects the user to the */login* page.
- Error Parsing: It extracts descriptive error messages from JSON responses, ensuring that the UI can display meaningful feedback to the user (e.g., "Seat no longer available" instead of a generic "500 Server Error").
- Network Resilience: The client distinguishes between application-level errors and infrastructure failures. It specifically detects *TypeError: Failed to fetch*, providing a clear notification if the API Gateway is unreachable or the user's network is disconnected.

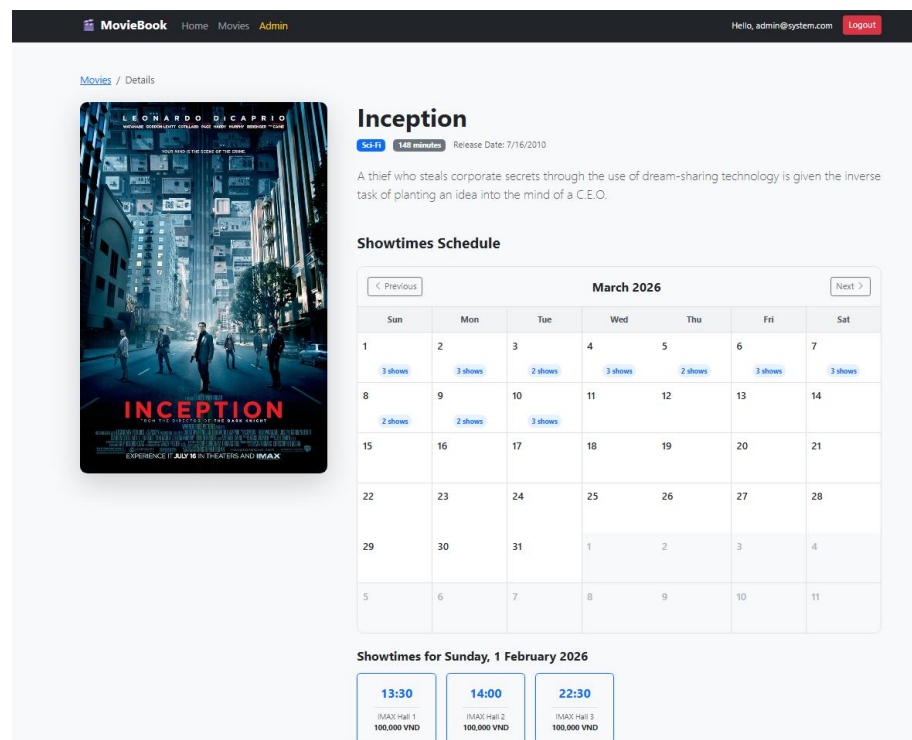
The module exports 27 specialized functions covering full system operations, including CRUD for movies, showtimes, bookings and payments method.

II.3 Customer-Facing Pages

II.3.1 Movies List & Detail

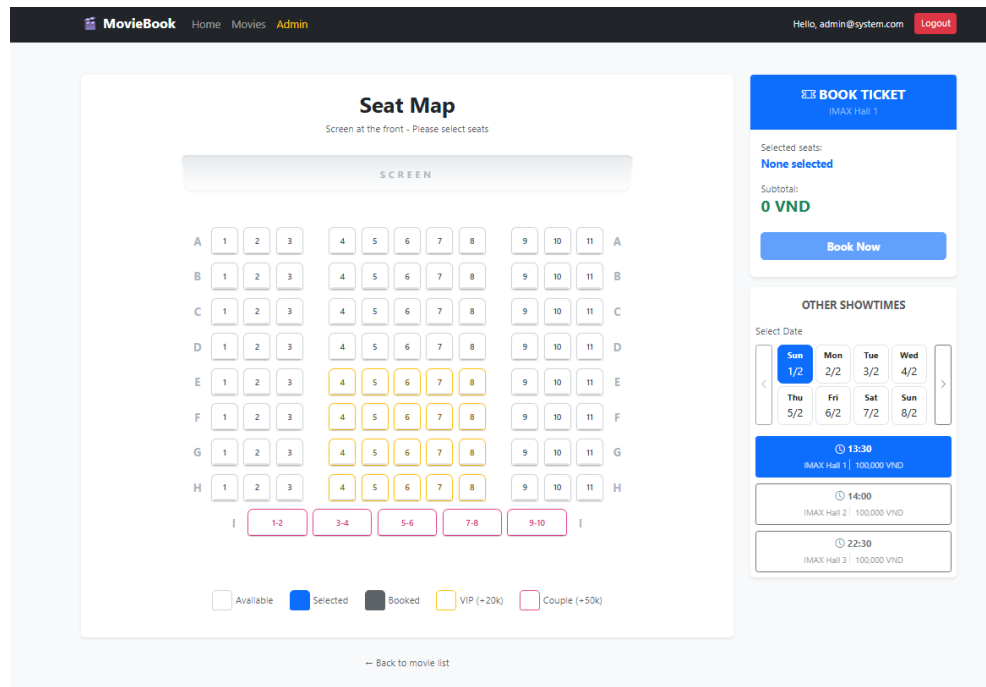


Movies List: displays a responsive 4-column grid of movie cards. It features a debounced search mechanism that triggers a query to the Movie Service after a 500ms delay in typing.



Movie Detail: Featuring a sticky movie poster and detailed metadata (genre, duration, description). Added interactive calendar and time grid for showtimes selection.

II.3.2 Core Booking Logic



The booking interface, technically, is the most complex component of the front end, featuring:

- **Dynamic Seat Map:** Renders seats row-by-row with specialized logic for aisle spacing and seat types. Seats are color-coded and styled based on their status and type.
- **Multi-seat Selection:** Users can toggle multiple seats, which are stored in a local state array. The system automatically calculates a running total by adding per-seat surcharges (e.g., +20k for VIP, +50k for Couple) to the base showtime price.
- **Schedule Picker:** A secondary navigation sidebar allows users to quickly switch between different showtimes for the same movie without leaving the booking context. It uses a paginated date scroller (8 dates per page) and highlights the current showtime.

II.3.3 Payment & Post-Booking Workflow

The screenshot shows a 'Payment Gateway' modal window. At the top, it states 'You are paying for booking ID: #97023' and 'Total amount: 100,000 VND'. Below this, there is a section 'Select payment method:' with a radio button selected for a card payment. The card details are masked as '**** * 3333' with the name 'ASD' below. A green 'Confirm Payment' button is prominently displayed. At the bottom, there is a link '+ Add / Use New Card'.

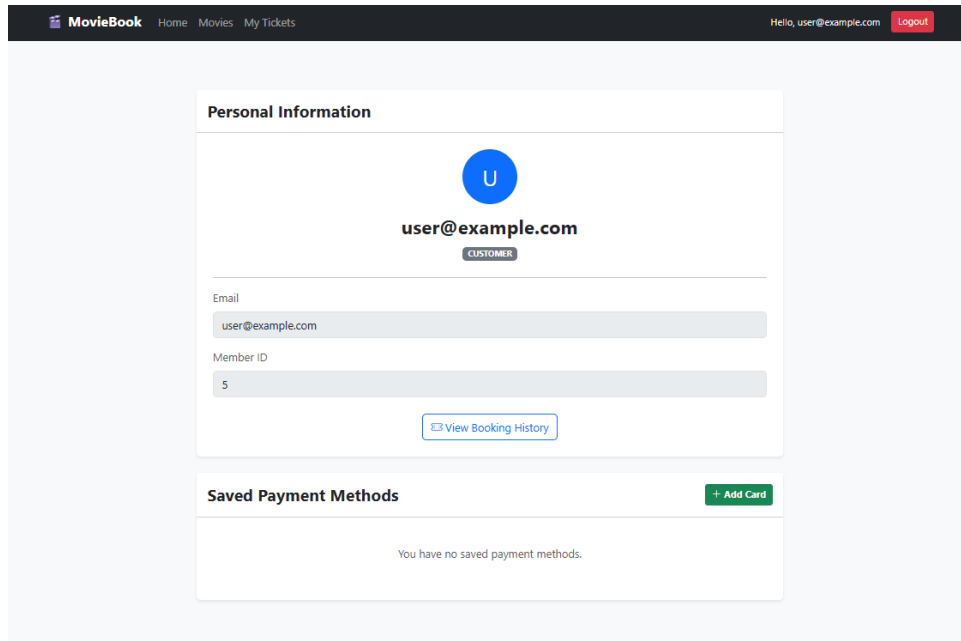
Payment Gateway: Manages two distinct states based on the user's history. New users are prompted to add a card via a Bootstrap modal, while returning users see a radio-select list of their Saved Payment Methods. Upon confirmation, the page utilizes *Promise.all* to process multiple booking IDs simultaneously before redirecting to the user's history.

The screenshot shows the 'My Booked Tickets' page. It features a table with columns: Booking ID, Showtime, Seats, Price, Status, and Actions. The table lists 17 bookings. Most are in a 'Paid' status, while the last one (#1174) is 'Pending Payment' and has a 'Pay' button. Each row also has a 'Cancel Ticket' button.

Booking ID	Showtime	Seats	Price	Status	Actions
#18	ID: 11	F6,D3	220,000 VND	Paid	Cancel Ticket
#19	ID: 12	G10,F9	200,000 VND	Paid	Cancel Ticket
#94	ID: 33	C8,B8	100,000 VND	Paid	Cancel Ticket
#152	ID: 65	C2,H10,E9	300,000 VND	Paid	Cancel Ticket
#184	ID: 74	E4	70,000 VND	Paid	Cancel Ticket
#210	ID: 81	B7	100,000 VND	Paid	Cancel Ticket
#216	ID: 81	G5,D2	220,000 VND	Paid	Cancel Ticket
#244	ID: 88	C10,B1	100,000 VND	Paid	Cancel Ticket
#317	ID: 107	G3,G10	200,000 VND	Paid	Cancel Ticket
#356	ID: 122	F1,H6	220,000 VND	Paid	Cancel Ticket
#437	ID: 148	C2,B1,H4	320,000 VND	Paid	Cancel Ticket
#653	ID: 211	F10	100,000 VND	Paid	Cancel Ticket
#813	ID: 261	E6,E4	240,000 VND	Paid	Cancel Ticket
#1034	ID: 323	H7,G1	220,000 VND	Paid	Cancel Ticket
#1174	ID: 372	C2,H11,C10	300,000 VND	Pending Payment	Pay Cancel Ticket

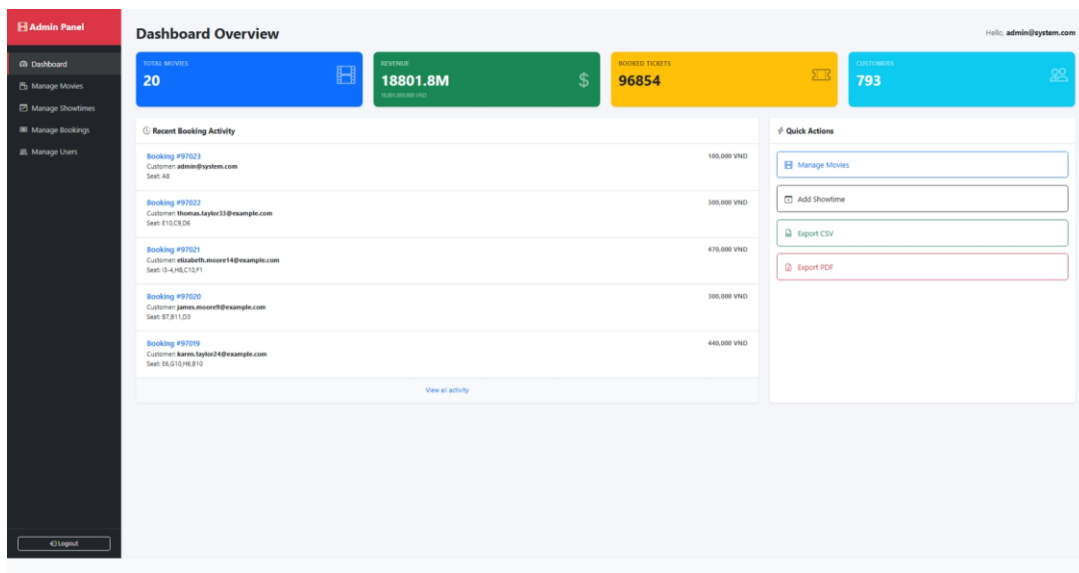
Booking Management: Provides a table of the user's transactions. It uses status badges ("Paid" vs "Pending Payment") and offers contextual actions: "Pay" for pending

reservations and "Cancel Ticket" that triggers a confirmation dialog and coordinates with the Booking Service to release seats.



User Profile: Serves as basic account hub, featuring full CRUD interface for payment methods.

II.4 Admin Dashboard & Management Pages



Dashboard:

- 4 stat cards (Total Movies, Revenue, Booked Tickets, Customers) fetched via *Promise.all*
- Recent Booking Activity list (last 5 bookings, sorted by ID desc)
- Quick actions panel with links to management pages.

- CSV/PDF export (using *jsPDF+jspdf / autotable*).

Poster	Title	Genre	Duration	Release Date	Revenue (YND)	Actions
	Inception	Sci-Fi	148 minutes	7/16/2010	978,646,000	Edit Delete
	The Dark Knight	Action	152 minutes	7/18/2008	981,300,000	Edit Delete
	Interstellar	Adventure	169 minutes	11/7/2014	658,500,000	Edit Delete
	The Matrix	Sci-Fi	136 minutes	3/31/1999	997,120,000	Edit Delete
	Avengers: Endgame	Action	181 minutes	4/26/2019	944,190,000	Edit Delete
	Parasite	Drama	132 minutes	5/30/2019	938,910,000	Edit Delete
	Spirited Away	Animation	125 minutes	7/20/2001	932,740,000	Edit Delete
	The Godfather	Crime	175 minutes	3/24/1972	903,070,000	Edit Delete
	Pulp Fiction	Crime	154 minutes	10/14/1994	941,940,000	Edit Delete
	Forrest Gump	Drama	142 minutes	7/6/1994	993,900,000	Edit Delete
	Dune: Part Two	Sci-Fi	166 minutes	3/10/2024	907,450,000	Edit Delete
	Oppenheimer	Biography	180 minutes	7/21/2023	943,420,000	Edit Delete
	Barbie	Comedy	114 minutes	7/21/2023	917,310,000	Edit Delete
	Spider-Man: Across the Spider-Verse	Animation	140 minutes	6/2/2023	918,280,000	Edit Delete

Manage Movies:

- Table with poster thumbnail, title, genre, duration, release date, per-movie revenue (computed by cross-referencing showtimes -> bookings)
- Add/Edit modal with poster upload: File input triggers *FileReader* preview; Image URL input clears file selection (and vice versa); file upload calls *uploadMoviePoster()* before saving movie data.

Movie	Start	End	Ticket Price (YND)	Revenue (YND)	Actions
The Dark Knight Standard Hall 2	09:00	11:32	100,000	14,760,000	Edit Delete
Schindler's List Standard Hall 2	09:00	12:15	50,000	5,370,000	Edit Delete
The Silence of the Lambs Basic Hall 1	09:45	11:43	100,000	9,300,000	Edit Delete
Avengers: Endgame Standard Hall 2	10:15	13:16	50,000	6,120,000	Edit Delete
Barbie Standard Hall 1	10:15	12:09	30,000	7,600,000	Edit Delete
The Matrix Basic Hall 2	10:30	12:46	100,000	14,770,000	Edit Delete
Pulp Fiction Standard Hall 1	10:30	13:04	50,000	10,730,000	Edit Delete
The Matrix Standard Hall 1	11:15	13:31	50,000	7,010,000	Edit Delete
Spirited Away Standard Hall 1	11:30	13:35	50,000	6,620,000	Edit Delete
Pulp Fiction Basic Hall 1	11:30	14:04	100,000	16,590,000	Edit Delete
Forrest Gump Standard Hall 2	11:30	13:52	50,000	8,290,000	Edit Delete
GoodFellas Basic Hall 2	11:30	13:56	100,000	10,300,000	Edit Delete
Interstellar Basic Hall 2	12:15	13:04	100,000	8,390,000	Edit Delete
The Shawshank Redemption Standard Hall 1	12:30	14:52	50,000	8,510,000	Edit Delete
The Shawshank Redemption Standard Hall 2	13:00	15:52	30,000	4,910,000	Edit Delete
Dune: Part Two Basic Hall 1	13:45	16:31	100,000	10,800,000	Edit Delete

Manage Showtimes:

- Date filter: Pre-set to today, filters showtimes by *start_time* prefix.
- Table shows movie title, room name, start/end times, ticket prices and per-showtime revenue.
- Add modal: Movie and Room *<select>* dropdowns populated from API; Auto-calculates *end_time* from movie duration.
- Edit pre-fills date/time fields by splitting “YYYY-MM-DD HH:MM” string.

Admin Panel

Dashboard

Manage Movies

Manage Showtimes

Manage Bookings

Manage Users

Logout

Manage Bookings

Search by email...

ID	User Email	Showtime	Seats	Amount	Status	Actions
#103479	joseph.garcia15@example.com User ID: N/A	Showtime #6	67	180,000 vHD	Completed	Cancel
#103478	sarah.miller24@example.com User ID: N/A	Showtime #6	84	180,000 vHD	Completed	Cancel
#103477	patricia.hernandez17@example.com User ID: N/A	Showtime #6	908	100,000 vHD	Completed	Cancel
#103476	james.anderson3@example.com User ID: N/A	Showtime #6	FLA2,ELA3	420,000 vHD	Completed	Cancel
#103475	karen.brown17@example.com User ID: N/A	Showtime #6	GLA1,ELA2	420,000 vHD	Completed	Cancel
#103474	barbara.lopez16@example.com User ID: N/A	Showtime #6	74	120,000 vHD	Completed	Cancel
#103473	thomas.wilson19@example.com User ID: N/A	Showtime #6	HD,SLA	320,000 vHD	Completed	Cancel
#103472	mary.martin19@example.com User ID: N/A	Showtime #6	81,HL,88	320,000 vHD	Completed	Cancel
#103471	michael.taylor43@example.com User ID: N/A	Showtime #6	81	180,000 vHD	Completed	Cancel
#103470	mary.anderson16@example.com User ID: N/A	Showtime #18	83	70,000 vHD	Completed	Cancel
#103469	linda.davis42@example.com User ID: N/A	Showtime #18	810,SLA	180,000 vHD	Completed	Cancel
#103468	jennifer.rodriguez29@example.com User ID: N/A	Showtime #18	HD,SLA,SLA	370,000 vHD	Completed	Cancel
#103467	elizabeth.moore36@example.com User ID: N/A	Showtime #18	HL,SLA	140,000 vHD	Completed	Cancel
#103466	jessica.moore27@example.com User ID: N/A	Showtime #18	AL,SLA,SLA	210,000 vHD	Completed	Cancel
#103465	patricia.davis40@example.com User ID: N/A	Showtime #18	SLA	70,000 vHD	Completed	Cancel
#103464	charles.rodriguez39@example.com User ID: N/A	Showtime #18	SLA	70,000 vHD	Completed	Cancel

© 2025 www.360example.com

Manage Bookings:

- Full booking list sorted by ID descending with email search filter
- Cancel with confirmation for each.

Admin Panel

Dashboard

Manage Movies

Manage Showtimes

Manage Bookings

Manage Users

Logout

Manage Users

Search by email...

ID	Email	Role	Status	Actions
#1	admin@system.com	ADMIN	Active	Edit
#2	test_customer_TestGateway@example.com	TESTCUSTOMER	Active	Edit
#3	user_8d6115ee@test.com	TESTCUSTOMER	Active	Edit
#4	duplicate_TestFrone@example.com	TESTCUSTOMER	Active	Edit
#5	user@example.com	TESTCUSTOMER	Active	Edit
#6	susan.jones0@example.com	TESTCUSTOMER	Active	Edit
#7	david.johnson1@example.com	TESTCUSTOMER	Active	Edit
#8	joseph.smith2@example.com	TESTCUSTOMER	Active	Edit
#9	david.thomas3@example.com	TESTCUSTOMER	Active	Edit
#10	charles.lopez4@example.com	TESTCUSTOMER	Active	Edit
#11	michael.garcia5@example.com	TESTCUSTOMER	Active	Edit
#12	jessica.rodriguez6@example.com	TESTCUSTOMER	Active	Edit
#13	patricia.williams7@example.com	TESTCUSTOMER	Active	Edit
#14	jennifer.moore8@example.com	TESTCUSTOMER	Active	Edit
#15	joseph.johnson9@example.com	TESTCUSTOMER	Active	Edit
#16	thomas.johnson10@example.com	TESTCUSTOMER	Active	Edit
#17	jennifer.moore11@example.com	TESTCUSTOMER	Active	Edit
#18	joseph.gonzalez12@example.com	TESTCUSTOMER	Active	Edit
#19	jehu.martinez13@example.com	TESTCUSTOMER	Active	Edit
#20	elizabeth.jones14@example.com	TESTCUSTOMER	Active	Edit
#21	charles.rodriguez15@example.com	TESTCUSTOMER	Active	Edit
#22	patricia.garcia16@example.com	TESTCUSTOMER	Active	Edit

Manage Users:

- User table with role badges and email search
- Edit modal: Change role and optionally reset password.

II.5 Reusable Components

II.5.1 Global Navigation

```
Header = {
  render: () => {
    // 1. Fetch Session from LocalStorage
    user = localStorage.get('user');

    return `
      <nav>
        <a href="/">🎬 MovieBook</a>

        <!-- 2. Conditional Role Navigation -->
        IF user.role = 'admin'    => SHOW '<a href="/admin">Admin Panel</a>'
        IF user.role = 'customer' => SHOW '<a href="/tickets">My Tickets</a>'

        <!-- 3. Authentication Toggle -->
        IF NOT user:
          SHOW [Login Button], [Admin Login Button]
        ELSE:
          SHOW [Profile: user.email], [Logout Button]
      </nav>`;
  },
  afterRender: () => {
    // 4. Session Termination (Logout)
    ON click('#logout-btn'):
      localStorage.remove('user'); // Clear Session
      redirect('/');               // Reset to Home
  }
}
```

Header: Responsive Bootstrap navbar with conditional links based on role(**My tickets** for customers, **Admin** for administrators), logout button clears *localStorage*.

```
AdminSidebar = {
  render: (activePage) => {
    // 1. Sidebar Structure with Dark Theme
    return `
      <div class="admin-sidebar">
        <header>🎬 Admin Panel</header>

        <!-- 2. Dynamic Highlighting Logic -->
        <nav>
          LINK "Dashboard"    => ACTIVE IF activePage = 'dashboard'
          LINK "Movies"       => ACTIVE IF activePage = 'movies'
          LINK "Showtimes"    => ACTIVE IF activePage = 'showtimes'
          LINK "Bookings"     => ACTIVE IF activePage = 'bookings'
          LINK "Users"        => ACTIVE IF activePage = 'users'
        </nav>

        <!-- 3. Protected Session Termination -->
        <div class="user-footer">
          <button id="admin-logout-btn">Logout</button>
        </div>
      </div>`;
  },
  afterRender: () => {
    // 4. Logout with Safety Confirmation
    ON click('#admin-logout-btn'):
      IF confirm('Logout?'):
        localStorage.remove('user'); // Terminate Admin Session
        redirect('/');               // Reset to Public Home
  }
}
```


AdminSidebar: Dark sidebar with icon nav links (Dashboard, Movies, Showtimes, Bookings, Users), active state highlighting, logout with confirmation.

III. Dockerization

III.1 Docker images

Splitted into two Dockerfiles, managed via *docker-compose.yml*:

- *Dockerfile.backend*: Debian-based. Using Python 3.9 slim base to install packages via *requirements.txt*; Copies *src/*, used by all 6 backend services + *db_seeder* container. Each service specifies its own *command*: in *docker-compose.yml*.
- *Dockerfile.frontend*: Node 22 Alpine base – installs npm dependencies, runs Vite dev server on port 5173.

III.2 Docker Compose Orchestration

The *docker-compose.yml* defines 9 containers with startup order:

- RabbitMQ Health-First: Message broker server starts first. It includes a health check with a 10s interval and 5 retries. All backend services are configured to wait for the *service_healthy* state before attempting to connect.
- Tiered Bootstrapping: Core services (Movie, Booking, Payment, User and Notification) launch only after RabbitMQ is healthy. API Gateway is launched next, utilizing *service_started* condition for all four microservices.
- Frontend and database seeder start last. Ensuring the backend is ready.

Services communicate via Docker's internal DNS. Discovery is handled through environment variables, allowing routing without hardcoding IP addresses.

III.3 Hot-Reload

Source code volumes are mounted into every container.

```
volumes:
  - ./src:/app/src      # Backend source code
  - ./tests:/app/tests  # Test suite
  - ./logs:/app/logs    # Log output (movie_service)
```

For the frontend, the entire *./frontend* directory is mounted, but *node_modules* is preserved via an anonymous volume

```
volumes:
  - ./frontend:/app
  - /app/node_modules # Prevent host node_modules from overriding container's
```

III.4 Database Utilities

The *db_seeder* is a one-shot container that runs *seed_db.py* to populate all 4 databases with test data:

- 52 users: 1 admin + 1 default test user + 50 randomly generated customers
- 20 movies with real titles, poster filenames, descriptions and durations.
- ~2000+ showtimes: 2-3/day/movie over a 37-day window (past 30 days + future 7 days), distributed across 6 rooms with pricing logic
- Grouped bookings: For each showtime (70% chance), 1-20 random seats are booked, grouped into orders of 1-4 seats each with correct total amounts including surcharges.

reset_db.py uses *glob.glob* to find and delete all **.db* files under *src/**/db/*, with a hint to stop Docker containers first if files are locked.

IV. Testing

An end-to-end integration test that runs against live microservices processes. It uses Python's *unittest* framework with a shared *TestBase* class.

IV.1 Base Class

- Shared state via class-level *state* dict - stores *user_token*, *admin_token*, *movie_id*, *showtime_id*, *booking_id*, *card_id* across test methods.
- *_req()* helper: Sends HTTP requests with auto-injected key header, 3-attempt retry logic with *database is locked* detection (SQLite concurrency), colored console output, and configurable expected status code assertion.
- *get_real_available_seat()*: Fetches the seat map for showtime, filters to *status='available'*, and picks a random seat.
- *create_dedicated_showtime()*: Creates a fresh showtime via the Gateway (admin auth) to isolate test data.

IV.2 Test Orchestrator

- Step 1: Creates a *logs/* directory, then cleans all **.db** files (3 glob patterns covering root, *instance/*, and *src/**/*)
- Step 2: Launches all 6 services as subprocesses with **stdout/stderr** redirected to per-service log files (*logs/{name}_out.log*, *logs/{name}_err.log*) - this prevents buffered output from hanging the process.
- Step 3: Wait for each service port to accept TCP connections (socket polling, 45s timeout).
- Step 4: Runs test suites in logical order: Gateway → Notification → User → Movie → Booking → Payment → Errors.

- Step 5: Terminates all processes and closes log file handles on completion.

Note: It might run differently on Docker, but the order is still the same.

IV.3 Test Suites (6 modules, 32 test cases)

Test Module	Count	Key Scenarios Covered
Gateway Security	5	API Key check, RBAC (Admin vs. Customer), Header forwarding
User & Identify	4	Registration, Login, Profile retrieval, Admin user listing
Movie Management	7	CRUD Movies, Search, Room listing, Showtime creation
Booking	6	Grouped seats, Atomicity, My Bookings, Cascade release
Payment & Cars	5	Card CRUD, Process Payment, Payment for missing bookings
Notification	1	RabbitMQ Event production (OrderPlacedEvent)
Error Handling	4	404 (Missing Movie), 400 (Duplicates), 401 (Wrong Pass)